

```
In [11]: import numpy as np
from sklearn import datasets
from sklearn.dummy import DummyClassifier
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import KFold
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
In [12]: # Carga de datos.
iris = datasets.load_iris()
print(iris.target_names)
```

```
['setosa' 'versicolor' 'virginica']
```

```
In [13]: # Mostrar características de la tabla de datos.
print("Tabla de datos: %d instancias y %d atributos" % (iris.data.shape[0], iris
print("Valores de la clase:", set(iris.target))
```

```
Tabla de datos: 150 instancias y 4 atributos
```

```
Valores de la clase: {np.int64(0), np.int64(1), np.int64(2)}
```

```
In [14]: # Test: hold-out split 80-20%. PARTICIÓN EXTERNA
X_training, X_test, y_training, y_test= train_test_split(iris.data, iris.target,
valores_test, ocur_test= np.unique(y_test, return_counts=True)
print('Test: ', 'clases: ', valores_test, ' ocurrencias; ', ocur_test)
```

```
Test:  clases:  [0 1 2]  ocurrencias;  [10  9 11]
```

```
In [15]: # Estandarizar Las características de entrenamiento y de test
standardizer = StandardScaler()
X_train = standardizer.fit_transform(X_training)
X_test = standardizer.transform(X_test)
```

```
In [16]: # Definimos el algoritmo SVM para clasificación
from sklearn.svm import SVC
svc = SVC(C=0.5, gamma='auto')

# Hacemos el cross-validation interno para seleccionar Los mejores hiperparámetr
results = cross_val_score(svc, X_training, y_training, cv = KFold(n_splits=5)) #
print("Resultados por bolsa: ", results)
print("Accuracy (media +/- desv.): %0.4f +/- %0.4f" % (results.mean(), results.s
```

```
Resultados por bolsa:  [1.          0.95833333 0.875          0.95833333 0.95833333]
```

```
Accuracy (media +/- desv.): 0.9500 +/- 0.0408
```

```
In [17]: # Definimos el algoritmo SVM para clasificación
from sklearn.svm import SVC
svc = SVC(C=0.5, gamma='auto')

# Hacemos el cross-validation interno para seleccionar Los mejores hiperparámetr
results = cross_val_score(svc, X_training, y_training, cv = KFold(n_splits=5)) #
print("Resultados por bolsa: ", results)
print("Accuracy (media +/- desv.): %0.4f +/- %0.4f" % (results.mean(), results.s
```

```
Resultados por bolsa:  [1.          0.95833333 0.875          0.95833333 0.95833333]
```

```
Accuracy (media +/- desv.): 0.9500 +/- 0.0408
```

```
In [18]: # Una vez entrenado y validado el modelo para seleccionar Los mejores hyperparam
# "train" y "val" para entrenar el modelo definitivo
```

```
# svc = SVC(C=0.5, gamma='auto')
svc.fit(X_training, y_training)
```

Out[18]:

▼ SVC ⓘ ?

► Parameters

In [19]: *# Calcular la accuracy del conjunto de test*

```
test_results = svc.score(X_test, y_test)
print('Exactitud en test: ', test_results*100, '%')
```

Exactitud en test: 33.33333333333333 %

In [20]: *# Extraer las predicciones, en lugar de directamente la accuracy*

```
y_pred = svc.predict(X_test)
print('Predicciones:      ', y_pred)
print('Etiquetas reales: ', y_test)
```

```
Predicciones:      [2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 0 2 2 2 2 2]
Etiquetas reales:  [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]
```

In []: